MACHINE LEARNING

(22PC1AM302) — B.Tech V Semester

LECTURE NOTES

UNIT I: Introduction to Machine Learning and Pre-requisites

Syllabus scope — Unit I: Introduction to Machine Learning · Learning Paradigms · PAC Learning · Version Spaces · Role of Machine Learning in Artificial Intelligence Applications.

Unit Learning Outcomes (mapped to CO-1)

- Define machine learning and distinguish it from traditional programming.
- Describe the three learning paradigms — supervised, unsupervised and reinforcement learning.
- Explain the concept of Version Spaces and trace the Candidate-Elimination algorithm on a worked example.
- State the framework of PAC (Probably Approximately Correct) learning and reason about learnability.
- Identify the role and applications of machine learning within artificial intelligence.

1. Introduction to Machine Learning

1.1 What is Machine Learning?

Machine Learning (ML) is the field of study that gives computers the ability to improve their performance on a task through experience, without being explicitly programmed for every scenario. Instead of a human writing fixed rules, a learning algorithm is given data (examples of the task) and it automatically works out a model — a set of parameters — that captures the pattern in that data.

The term "Machine Learning" was coined by Arthur Samuel in 1959 while working at IBM on a program that learned to play checkers. There is no single universally accepted definition of the field; the most widely used operational definition is due to Tom Mitchell:

Definition: Machine Learning (Tom Mitchell, 1997)

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

This definition is useful because it forces us to be precise about three things whenever we set up a learning problem: the Task (T) we want the system to perform, the Performance measure (P) that tells us how well it is doing, and the Experience (E) — the data — that it learns from.

Example: Framing learning problems using T, P, E

Handwriting recognition — T: classify handwritten words in an image; P: percentage of words classified correctly; E: a labelled dataset of handwritten word images.

Autonomous driving — T: steer a vehicle using camera input; P: average distance travelled before a human intervention; E: recorded images and steering commands from a human driver.

Game playing (chess) — T: choose the next move; P: percentage of games won; E: games played against itself (self-play).

A program that learns from experience in this way is called a learning program or a learner.

1.2 Components of the Learning Process

Whether the learner is a human or a machine, the learning process can be broken down into four cooperating components:

1. Data storage — facilities to store and retrieve the large volumes of data the learner observes (a brain, or a disk/database).
2. Abstraction — extracting general concepts from stored raw data. Fitting a model to data (training) converts raw examples into a compact, abstract representation.
3. Generalization — turning the abstracted knowledge into a form that can act on new, unseen situations that are similar, but not identical, to the training data.
4. Evaluation — measuring how useful/accurate the learned knowledge is, and feeding that back to improve the cycle.

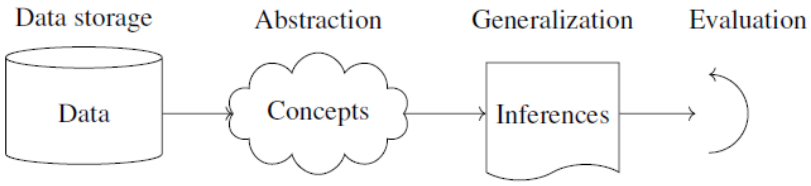


Figure 1.1: Components of learning process

Fig. 1.1 — Components of the learning process (Data storage → Abstraction → Generalization → Evaluation)

1.4 Applications of Machine Learning

Applying machine learning methods to large databases to extract useful, previously unknown patterns is called data mining. Typical application areas include:

- Retail & finance — analysing consumer behaviour, credit scoring, fraud detection, stock-market analysis.
- Manufacturing — process optimization, control and fault troubleshooting.
- Medicine — diagnosis support from patient data and medical images.
- Telecommunications — analysing call patterns for network optimization.
- Science — physics, astronomy and biology, where data volumes are too large for manual analysis.
- Vision, speech and robotics — object/speech recognition and robot control.

- Autonomous vehicles — steering models learned from human driving demonstrations.
- Games — programs that learn to play chess, backgammon, Go, etc.

2. Learning Paradigms

Machine learning algorithms are broadly organised into three paradigms based on the nature of the feedback available to the learner during training.

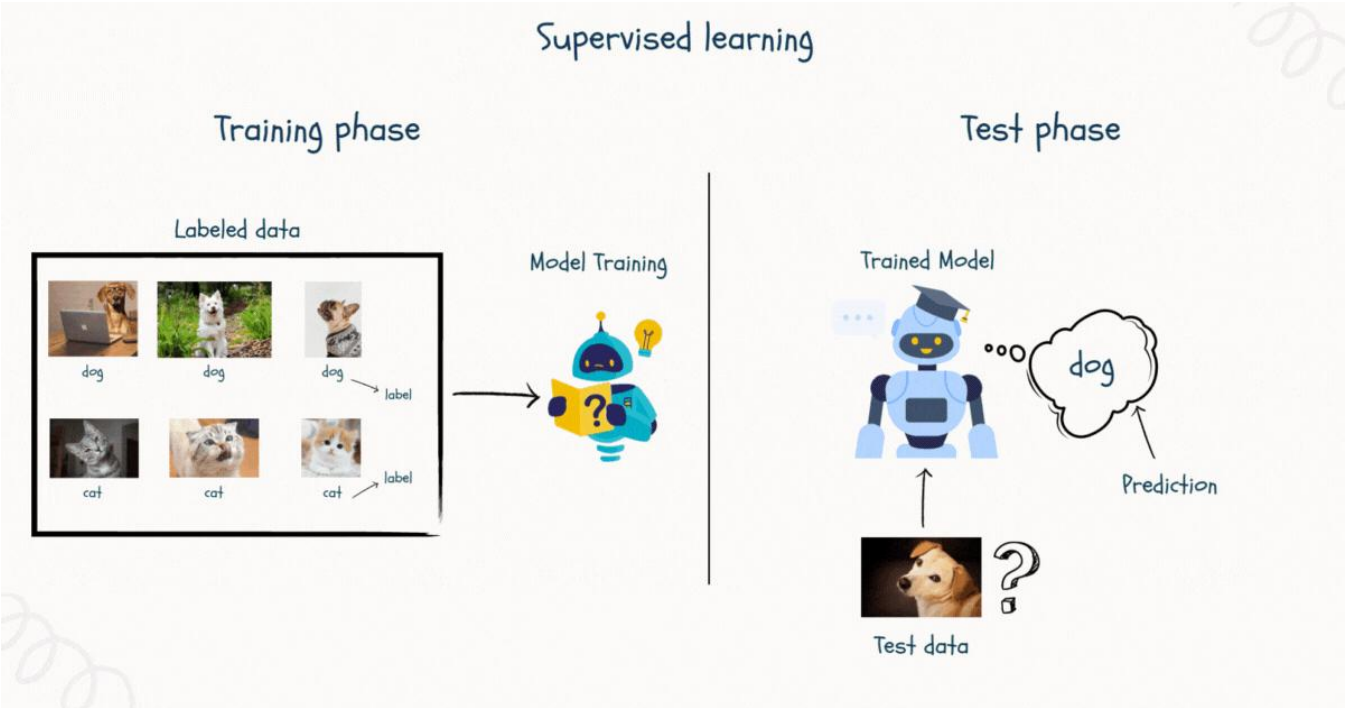
2.1 Supervised Learning

> **Definition: Supervised Learning**
> *A training set of labelled examples (input, correct output) is given, and the algorithm learns a function that generalizes this input-output mapping so it can predict the output for new, unseen inputs.*

Supervised learning is the machine learning task of learning a function that maps an input to an output based on example input-output pairs.
In supervised learning, each example in the training set is a pair consisting of an input object (typically a vector) and an output value. A supervised learning algorithm analyzes the training data and produces a function, which can be used for mapping new examples. In the optimal case, the function will correctly determine the class labels for unseen instances. Both classification and regression problems are supervised learning problems.
A wide range of supervised learning algorithms are available, each with its strengths and weaknesses. There is no single learning algorithm that works best on all supervised learning problems.
A "supervised learning" is so called because the process of algorithm learning from the training dataset can be thought of as a teacher supervising the learning process. We know the correct answers (that is, the correct outputs), the algorithm iteratively makes predictions on the training data and is corrected by the teacher. Learning stops when the algorithm achieves an acceptable level of performance.
Example :
Consider the following data regarding patients entering a clinic. The data consists of the gender and age of the patients and each patient is labelled as "healthy" or "sick"

gender	age	label
M	48	sick
M	67	sick
F	53	healthy
M	49	healthy
F	34	sick
M	21	healthy

> **Example: Supervised learning**
> Patients arriving at a clinic are described by gender and age, and each is labelled "healthy" or "sick". A model trained on such labelled records can predict the health status of a new patient.



Supervised Learning use cases

Some common use cases for supervised learning

1. Image classification: Identifying objects or features within an image, such as recognizing handwritten digits or determining whether an image contains a certain type of animal.
2. Text classification: Categorizing text into different classes, such as determining whether an email is spam or not.
3. Natural Language Processing (NLP): Tasks such as language translation, text summarization, and sentiment analysis.
4. Predictive modeling: Using historical data to make predictions about future events, such as stock prices or weather patterns.

5. Medical diagnosis : Identifying diseases from medical images, ECG or other vital signs
6. Object detection in videos and images : Identify and locate objects in videos and images.

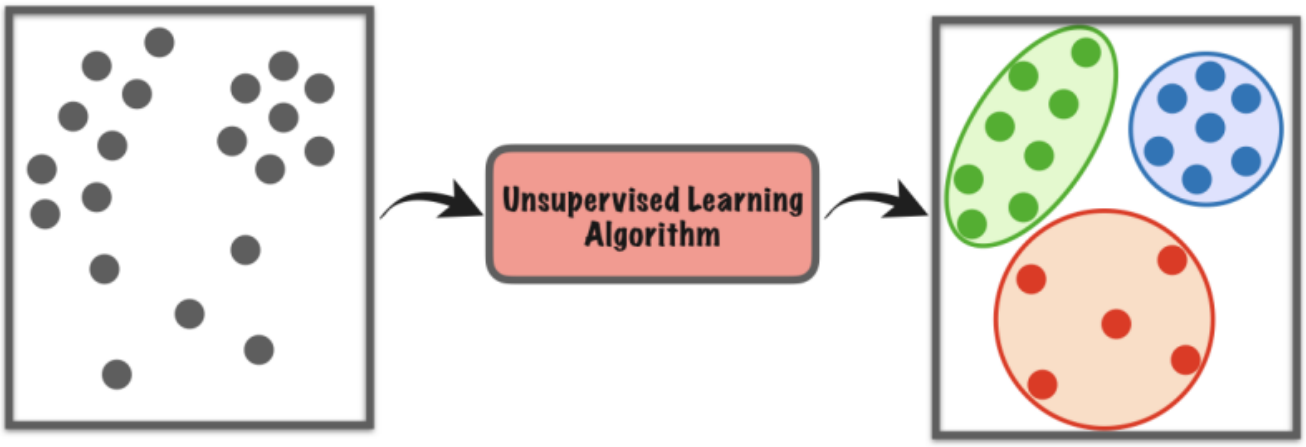
Supervised Learning algorithms

There are several popular supervised learning algorithms that are widely used in the field of machine learning. Some of the most well-known and commonly used algorithms include:

1. Linear Regression
2. Logistic Regression
3. Decision Trees
4. Random Forest
5. Support Vector Machines (SVM)
6. k-Nearest Neighbors (kNN)

2.2 Unsupervised Learning

> **Definition: Unsupervised Learning**
> *A type of learning where the training data has no labelled responses; the algorithm instead looks for structure — similarities, patterns or groupings — among the inputs themselves.*



Because there is no target output, there is no notion of prediction error to minimise, and the accuracy of the discovered structure cannot be evaluated the way a supervised model can. The most common technique is cluster analysis, used for exploratory analysis to discover hidden groupings in data. The statistical name for this general approach is density estimation.

Unsupervised Learning use cases

Unsupervised learning is used for a variety of tasks, including:

1. Clustering: Grouping similar data points together, such as grouping customers based on their purchasing patterns.
2. Dimensionality reduction: Reducing the number of features in a dataset, such as identifying the most important features in a set of images.
3. Anomaly detection: Identifying unusual or abnormal data points, such as detecting fraud in financial transactions.
4. Generative models: Creating new data that is similar to the input data, such as generating new images or text.

Unsupervised Learning algorithms

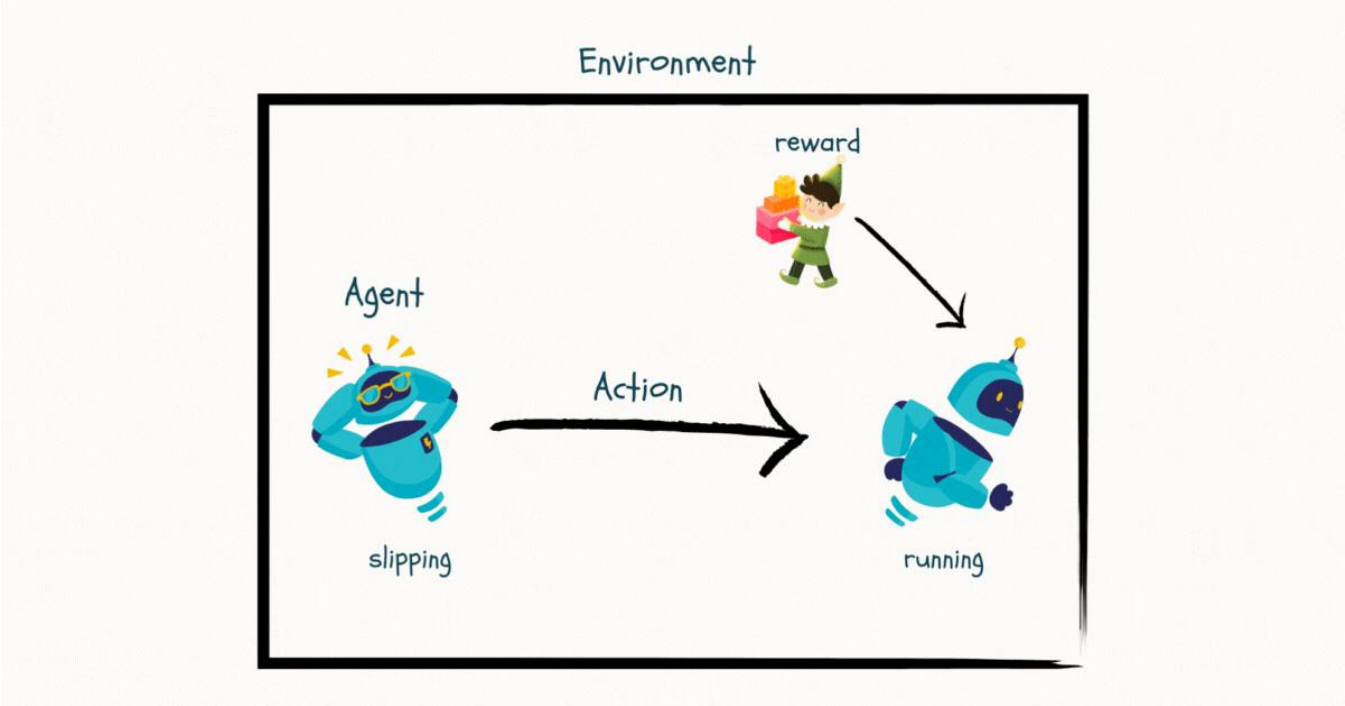
There are several popular unsupervised learning algorithms:

1. K-Means
2. Hierarchical Clustering
3. PCA(Principal Component Analysis)

2.3 Reinforcement Learning

Definition: Reinforcement Learning
A learning paradigm in which an agent interacts with an environment and learns, by trial and error, which sequence of actions maximizes a cumulative numerical reward — without being told directly which action is correct at each step.

Reinforcement learning sits conceptually between supervised and unsupervised learning: the agent is told when a result is wrong (a form of feedback, unlike unsupervised learning), but is never told the correct action to take (unlike supervised learning). It must explore different actions and learn from the reward/penalty signal — hence it is sometimes called “learning with a critic”. Because actions can affect not just the immediate reward but also future situations and rewards, RL is fundamentally a sequential decision-making problem.



Example: Reinforcement learning

Training a dog: you cannot tell it exactly what to do, but you can reward or scold it after it acts. Over repeated trials it learns which behaviours earn a reward. The same idea is used to train agents that play backgammon or chess, schedule jobs, or control robot limbs.

Reinforcement Learning use cases

Reinforcement learning is used in a variety of tasks, including:

1. Robotics: Training robots to perform specific tasks, such as grasping objects or moving through a maze.
2. Game playing: Training agents to play games such as chess, Go, and poker at a superhuman level.
3. Autonomous vehicles: Training self-driving cars to navigate through traffic and make safe driving decisions.

4. Industrial control: Optimizing the control of industrial systems, such as power plants or manufacturing processes.
5. Healthcare: Developing personalized treatment plans for patients based on their medical history and response to treatment.
6. Finance: Developing trading strategies and portfolio management.

Reinforcement Learning algorithms

There are several popular reinforcement learning algorithms:

1. Q-Learning
2. SARSA
3. DQN
4. A3C

2.4 Comparing the Three Paradigms

Aspect	Supervised	Unsupervised	Reinforcement
Feedback	Correct label given for every example	No labels provided	Reward/penalty signal, not the correct action
Goal	Learn input → output mapping	Discover structure/groupings	Learn a policy that maximizes cumulative reward
Typical tasks	Classification, Regression	Clustering, Dimensionality reduction	Sequential decision making, control
Example	Spam vs. not-spam email	Customer segmentation	Game playing, robot control

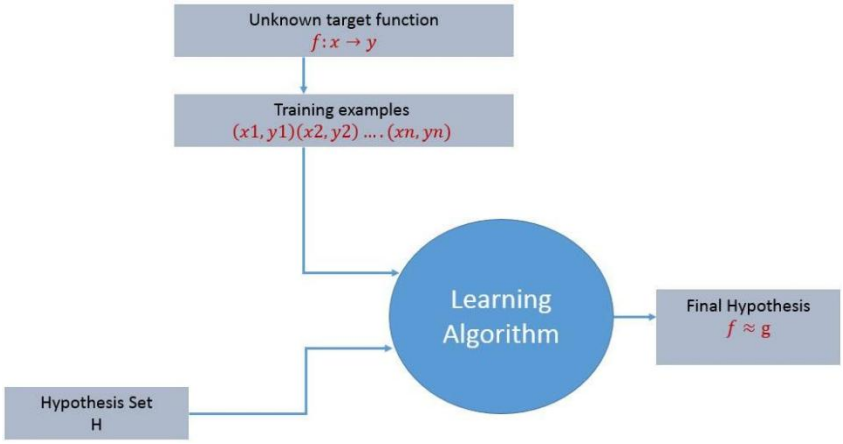
3. Version Spaces

Think of it like this: you're teaching a computer to recognize something — say, "what kind of weather is good for playing outside?" You show it a few examples (some good days, some bad days), and it has to figure out the RULE that separates good days from bad days.

That's it. That's concept learning. The computer is learning a concept (a rule) from examples.

Candidate Elimination Algorithm (CEA) is just one way for the computer to figure out that rule — and instead of guessing one rule, it keeps track of ALL the rules that could still be correct, and narrows them down as it sees more examples.

Version spaces provide a way to reason about concept learning as a search through a space of hypotheses, keeping track of exactly the hypotheses that remain consistent with the training examples seen so far — without listing them all individually.



Concept learning as search: a learning algorithm searches hypothesis set H, guided by training examples, to output a final hypothesis $g \approx f$.

Term	What it Really Means
Concept Learning	Computer learning a rule from examples (like learning "what makes a good day for sports")
Hypothesis	A guess/rule — e.g., "if it's Sunny and Warm, it's a good day"
General Hypothesis (G)	A very loose, "accepts almost anything" guess. Written with '?' — means "I don't care about this feature"
Specific Hypothesis (S)	A very strict, "only accepts exact matches" guess. Written with exact values
Version Space	All the rules that are STILL possibly correct, sitting between S and G
Positive Example	A "Yes" example — the concept IS true here
Negative Example	A "No" example — the concept is NOT true here

3.1 Basic Definitions

Definition: Consistent hypothesis

A hypothesis h is consistent with a set of training examples D if and only if h(x) = c(x) for every example (x, c(x)) in D, where c is the true target concept.

It is important to distinguish consistency from satisfaction: an instance x satisfies a hypothesis h whenever h(x) = 1, regardless of whether x is actually a positive or negative example of the target concept; whereas x is consistent with h only when h(x) equals the true label c(x).

📌 Definition: Version Space (VS_H,D)

The version space with respect to hypothesis space H and training data D is the subset of all hypotheses in H that are consistent with every example in D. A concept is complete if it covers all positive examples, and consistent if it covers none of the negative examples; the version space is the set of all complete and consistent hypotheses.

3.2 The LIST-THEN-ELIMINATE Algorithm

The most direct (but impractical) way to compute the version space is to start with every hypothesis in H and discard the ones that misclassify any training example:

1. Initialize VersionSpace to the list of every hypothesis in H.
2. For each training example (x, c(x)), remove from VersionSpace any hypothesis h for which $h(x) \neq c(x)$.
3. Output the list of hypotheses remaining in VersionSpace.

This works correctly whenever H is finite, but it requires exhaustively enumerating every hypothesis in H, which is infeasible for all but the smallest hypothesis spaces.

3.3 A Compact Representation: the S and G Boundaries

Definition: General boundary G

The set of maximally general hypotheses in H that are consistent with D — no hypothesis in H that is more general than a member of G is still consistent with D.

Definition: Specific boundary S

The set of maximally specific (minimally general) hypotheses in H that are consistent with D — no hypothesis in H that is more specific than a member of S is still consistent with D.

The Version Space Representation Theorem states that, for well-defined S and G, the version space equals exactly the set of hypotheses h such that some s in S is more specific than or equal to h, and some g in G is more general than or equal to h. In other words, S and G act like the "floor" and "ceiling" that fully bracket the version space.

3.4 CANDIDATE-ELIMINATION Algorithm

The CANDIDATE-ELIMINATION algorithm computes the version space incrementally by updating only the S and G boundaries as each training example is processed, avoiding the need to enumerate H explicitly.

Initialize G to the set of maximally general hypotheses in H, and S to the set of maximally specific hypotheses in H. For each new training example d:

The Candidate Elimination Algorithm updates the version space after processing each training example.

Step 1: Load the training dataset.

Step 2: Initialize General boundary G,Specific boundary S

Step 3: Process each training example one at a time.

Step 4: If the training example is positive:

Remove hypotheses from G that are inconsistent with the example.

Generalize S whenever necessary so that it covers the positive example.

Step 5: If the training example is negative:

Remove hypotheses from S that incorrectly classify the negative example.

Specialize G so that it excludes the negative example while remaining consistent with all positive examples.

Step 6: Repeat the process until all training examples have been processed.

The remaining hypotheses between S and G form the final version space.

If d is a positive example

- Remove from G any hypothesis inconsistent with d.
- For each hypothesis s in S that is inconsistent with d: remove s, and add all minimal generalizations of s that are (a) consistent with d and (b) more specific than some member of G.
- Remove from S any hypothesis that is more general than another hypothesis in S.

If d is a negative example

- Remove from S any hypothesis inconsistent with d.
- For each hypothesis g in G that is inconsistent with d: remove g, and add all minimal specializations of g that are (a) consistent with d and (b) more general than some member of S.
- Remove from G any hypothesis that is less general than another hypothesis in G.

3.5 Worked Example — "EnjoySport"

- Step 1: Start with G = "accept everything" and S = "accept nothing yet".
- Step 2: Look at one training example at a time.
- Step 3: If it's a POSITIVE (Yes) example → make S a little more general so it now covers this example too. Also remove any rule in G that would wrongly reject this example.
- Step 4: If it's a NEGATIVE (No) example → make G a little more strict so it excludes this example. Also remove any rule in S that would wrongly accept this example.

Consider the classic EnjoySport concept-learning task: each day is described by six attributes (Sky, AirTemp, Humidity, Wind, Water, Forecast), and the label indicates whether a person enjoys their favourite water sport that day.

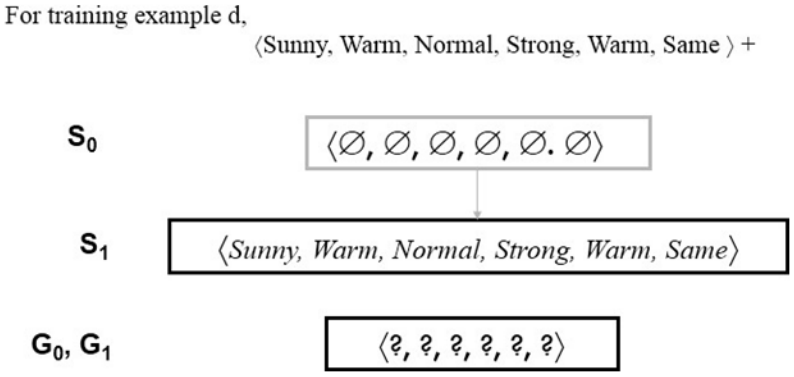
A Concept Learning Task – Enjoy Sport
Training Examples

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	YES
2	Sunny	Warm	High	Strong	Warm	Same	YES
3	Rainy	Cold	High	Strong	Warm	Change	NO
4	Sunny	Warm	High	Strong	Warm	Change	YES

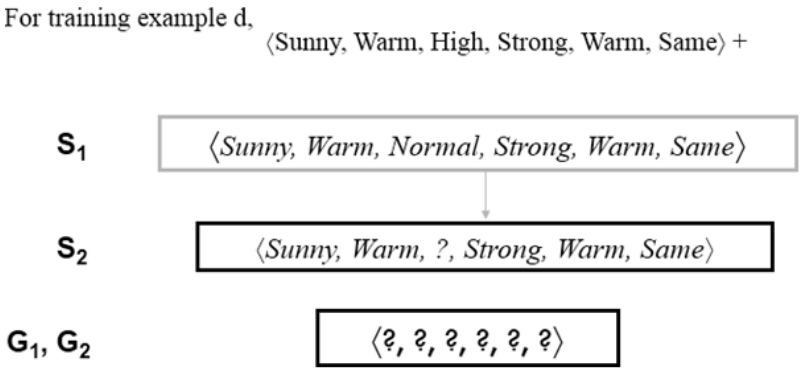
ATTRIBUTES CONCEPT

Boundaries are initialised to the most general hypothesis G0 = ⟨?, ?, ?, ?, ?, ?⟩ and the most specific hypothesis S0 = ⟨∅, ∅, ∅, ∅, ∅, ∅⟩, and then updated example by example:

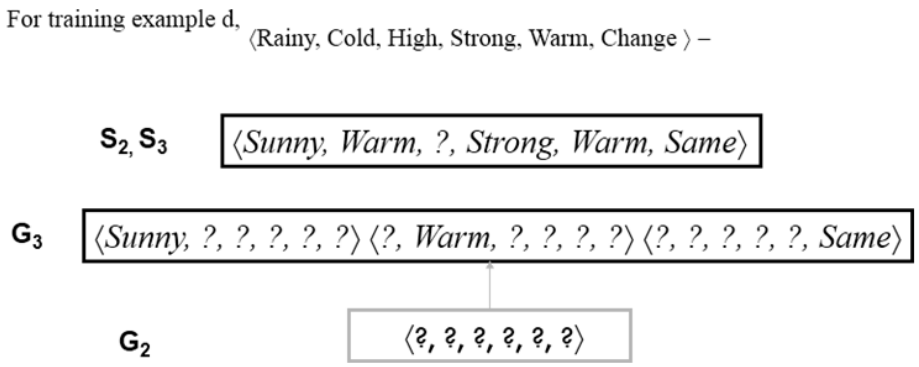
Example 1 (positive): S is too specific and fails to cover it, so S is generalised minimally to match; G0 already covers it, so G is unchanged.



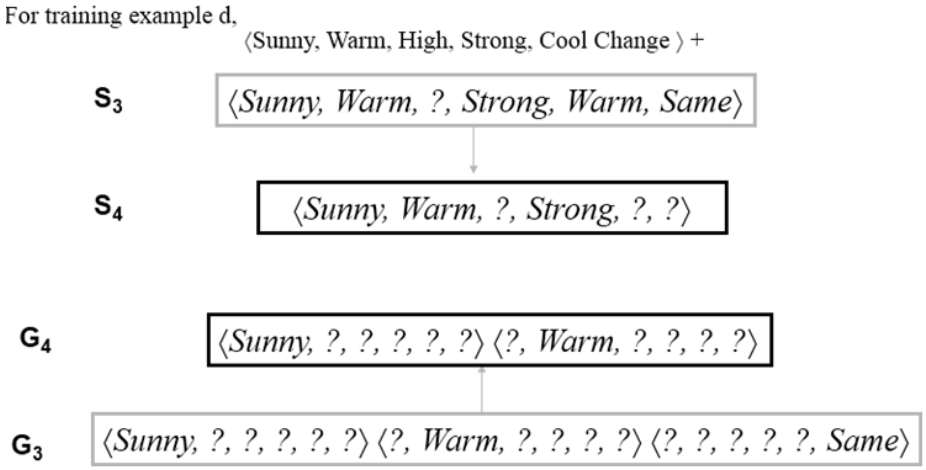
Example 2 (positive): S is generalised further (dropping the Humidity constraint); G remains unchanged.



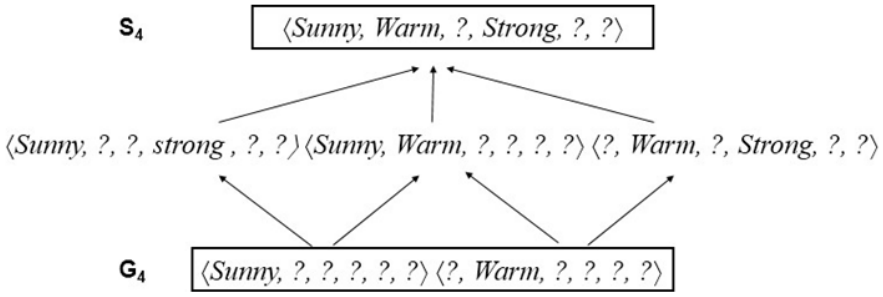
Example 3 (negative): G0 is now too general — it wrongly predicts "yes" — so G is specialised into three alternative hypotheses, each ruling out this negative example while staying consistent with the positive examples already seen.



Example 4 (positive): S is generalised again (dropping the Water/Forecast constraints), and any member of G that fails to cover this positive example is removed.



After processing all four examples, the resulting S4 and G4 boundaries jointly delimit the final version space — every hypothesis consistent with all four training examples lies between them, as shown below.



The final version space for EnjoySport: every hypothesis reachable from G4 down to S4 is consistent with all four training examples.

4. Probably Approximately Correct (PAC) Learning

- **PAC** = Probably Approximately Correct
- Introduced by Leslie Valiant (1984)
- A theory that tells us: when can an algorithm 'learn' a rule from examples and trust it will work on new, unseen data?
- **Probably** → how confident we are the answer is correct
- **Approximately Correct** → how accurate the answer is (doesn't need to be perfect, just close enough)

Simple way to think about it: PAC learning answers — "If I give a learning algorithm enough examples, can it find a good enough answer, most of the time?"

Why is PAC Learning Important?

- Gives a mathematical guarantee that a model will work well on new data, not just training data
- Helps decide how much data is needed for good learning
- Foundation for designing and analyzing ML algorithms
- Balances three things: accuracy, confidence, and amount of data (samples)

Term	Simple Meaning
Sample Complexity	How many examples/data points are needed to learn well
Hypothesis Space	The set of all possible models the algorithm can choose from
Generalization	How well the model performs on new, unseen data

Sample Complexity

- More accuracy needed → more samples needed
- More confidence needed → more samples needed
- More complex hypothesis space → more samples needed

Hypothesis Space

- Bigger space = more flexible, but higher risk of overfitting (memorizing training data)
- Smaller space = safer from overfitting, but less flexible
- Goal: balance flexibility vs. generalization

Generalization

- The real goal of ML — a model is only useful if it works on new data, not just training data

Overfitting vs. Underfitting

	Hypothesis Space Too Large	Hypothesis Space Too Small
Problem	Overfitting	Underfitting
What happens	Memorizes training data, fails on new data	Too simple, misses real patterns

PAC Insight	Needs way more samples to generalize	Needs fewer samples but may never be accurate enough

Golden rule: The right hypothesis space size = enough data to fit it well.

4. The PAC Learning Theorem (Formula)

$$m \geq \frac{1}{\varepsilon}\left(\log\frac{1}{\delta} + VC(H)\right)$$

Where:

- **m** = number of samples needed
- **ε (epsilon)** = desired accuracy (how small the error should be)
- **δ (delta)** = desired confidence (how sure we want to be)
- **VC(H)** = VC dimension — measures the complexity of the hypothesis space

In plain words: the more accuracy and confidence you want, and the more complex your model space is, the more training examples (m) you need.

Worked Example

- ε = 0.1 (want error ≤ 10%)
- δ = 0.05 (want 95% confidence)
- VC(H) = 3 (fairly simple hypothesis space)

Plugging these into the formula:

$$m \geq \frac{1}{0.1}\left(\log\frac{1}{0.05} + 3\right)$$

This gives the minimum number of training examples needed — PAC learning turns 'how much data do I need?' into solvable math.

VC Dimension

- VC dimension = how many points a model can perfectly separate/classify, no matter how they are arranged
- Example: a straight line in 2D can perfectly classify any arrangement of 3 points → VC dimension = 3
- It cannot always separate 4 points correctly → so VC dimension stops at 3
- Higher VC dimension → more powerful model → but needs more data to avoid overfitting

Challenges of PAC Learning

Real-world Applicability

- Assumes a finite hypothesis space and a true underlying function — not always true in practice
- Real data distributions can be complex, unknown, or the hypothesis space may be infinite

Computational Complexity

- Finding the optimal hypothesis can be computationally expensive, especially for large/complex spaces
- Efficient algorithms are needed to make PAC learning practical

Model Assumptions

- Assumes the data distribution is known and the true function is inside the hypothesis space
- These assumptions may not hold in the real world

5. Role of Machine Learning in Artificial Intelligence Applications

Artificial Intelligence (AI) is the broader goal of building systems that behave intelligently; Machine Learning is the principal modern approach for achieving that goal — instead of hand-coding intelligent behaviour rule by rule, an AI system is given the ability to learn the required behaviour from data or experience.

5.1 Why AI needs Machine Learning

- Many real-world tasks (vision, speech, natural language) are too complex for a human to specify explicit rules for every situation; ML lets the system infer the rules from examples.
- ML allows an AI system to adapt to changes in its environment over time, so the designer does not need to anticipate every possible situation in advance.
- Learned models generalize to previously unseen inputs, which is essential for an AI system to operate reliably outside its exact training conditions.

5.2 Representative Application Areas

AI Application Area	Role of Machine Learning
Computer Vision	Learning to recognize objects, faces and scenes from labelled images
Speech & NLP	Learning acoustic and language models for recognition, translation and generation
Robotics	Learning control policies for manipulation, navigation and locomotion
Autonomous vehicles	Learning to steer and make driving decisions from sensor/demonstration data
Game playing AI	Learning strategies for chess, Go, backgammon etc. through self-play
Recommender & retail systems	Learning customer preferences to personalize recommendations
Healthcare AI	Learning diagnostic and prognostic models from patient data

AI Application Area	Role of Machine Learning
Finance	Learning models for credit scoring, fraud detection and market analysis